Based on the regression testing strategies outlined and the Omninet AI project context, here are concrete examples for each strategy applied to the Omninet system.

---

1. Traceability-Based Regression Testing

Strategy: Use requirements traceability to identify which test cases must be rerun when a specific requirement changes.

Omninet Example:

Element	Description
Change Made	The payment logic is updated to support a new digital wallet (e.g., Alipay) as required by MRD 3.1.2.
Traceability Link	Requirement MRD 3.1.2 traces to System Requirements 010-010-110 (Payment acceptance) and 070-020 (Payment Decision Table). These trace to specific test cases: TC-PAY-001 (Visa approval), TC-PAY-002 (Mastercard approval), TC-PAY-003 (Debit with PIN), TC-PAY-007 (Digital wallet).
Regression Test Selection	Rerun **all test cases linked to payment requirements** (approx. 15-20 test cases) to ensure existing payment methods still work after adding the new one.

Element	Description
Tests NOT Rerun	Tests for AI Assistant language translation (unrelated to payment) are skipped.

---

2. Change/Impact Analysis Regression Testing

Strategy: Analyze the code/design changes to determine which modules might be affected, then test those modules and their dependents.

Omninet Example:

Element	Description
Change Made	A bug fix in the **Session Manager** module that handles the 60-second warning popup (MRD 3.1.2). The fix modifies the timing logic.
Impact Analysis	The Session Manager is called by: (a) Payment Processor (when time is purchased), (b) Browser module (to check remaining time), (c) UI module (to display warning). Changes could affect session termination, time display, and the "Buy Time" prompt.

Element	Description
Regression Test Selection	Rerun tests that involve: • Session start/end • Time warning popup • Purchasing additional time • Automatic session termination at zero time
Tests NOT Rerun	AI Assistant conversation tests, language switching tests (unrelated to session timing).

---

3. Risk-Based Regression Testing

Strategy: Prioritize regression testing based on the risk of failure (business impact, technical complexity, historical defect density).

Omninet Example:

Risk Level	Feature Area	Regression Test Effort
High Risk	**Payment Processing** (MRD 3.1.2) - affects revenue directly; complex decision table; history of bugs in this area.	Rerun **all** payment test cases (100% regression).
Medium Risk	**AI Assistant** (MRD 3.1.3) - important for user experience but not revenue-critical; failures are visible but not catastrophic.	Rerun **core scenarios** (language detection, common queries) but skip edge cases.
Low Risk	**Welcome Screen Logos** (MRD 3.1.1) - purely cosmetic; no functional impact.	**No regression testing** - rely on visual inspection during smoke test.

Example Scenario:

- A patch is released that updates the **AI content filtering model** (MRD 3.1.6).
- **Risk analysis:** Content filtering is high-risk because blocking errors could harm user experience or legal compliance.
- **Regression selection:** Rerun all content filtering test cases (blocked sites, allowed sites, edge cases), but skip payment and UI tests.

---

4. Cross-Functional Tests for "Accidental Regression Testing"

Strategy: Use broad, end-to-end tests designed for other purposes (e.g., system testing, acceptance testing) to accidentally catch regressions in multiple areas.

Omninet Example:

Element	Description
Change Made	A minor UI update to the "Buy Time" button color (cosmetic change in CSS).
Cross-Functional Test Used	A **User Journey Test** originally designed for acceptance testing: *"User selects English, pays with credit card, browses three websites, asks AI Assistant for weather, buys more time, logs out."*
Accidental Regression Detection	Even though the change was only visual, the cross-functional test catches that the button is now **unresponsive** due to an accidental JavaScript error introduced in the same deployment.
Why This Works	The broad test covers many components (UI, payment, browser, AI) and reveals side effects that narrow unit tests might miss.

Another Example:

- After updating the **AI Orchestrator** microservice, a cross-functional **Performance Test** (originally for system testing) reveals that response times for payment processing have increased because the orchestrator is consuming too many CPU resources – an unintended side effect.

---

5. Code Coverage to Assess Risk Level

Strategy: Measure code coverage after initial testing. Areas with low coverage are higher risk and should be prioritized for regression testing.

Omninet Example:

Element	Description
Change Made	A patch fixes a bug in the **offline mode caching logic** (when kiosk loses connection to cloud).
Coverage Analysis	The offline mode module currently has **only 60% branch coverage** in the existing test suite (many error-handling paths untested).
Risk Assessment	High risk – changes in this area could easily break untested error paths.
Regression Test Selection	In addition to rerunning existing offline mode tests, the team **creates and runs new tests** to cover the previously untested branches (e.g., "database full during offline caching", "network reconnection during transaction").
Outcome	The new tests uncover a regression where the kiosk crashes if the local cache is full – a defect that would have slipped into production.

Summary Table

Regression Strategy	Omninet Example Change	Tests Selected for Regression
Traceability	Add Alipay support	All payment-related test cases
Change/Impact	Fix 60-second warning timer	Session management, time purchase, termination tests
Risk-Based	Update AI content filter	Full content filter regression; skip payment/UI
Cross-Functional	Change "Buy Time" button color	Run full user journey test (catches JavaScript error)
Code Coverage	Fix offline mode bug	Create new tests for untested error paths + rerun existing

These strategies ensure that Omninet's regression testing is **efficient** (not rerunning everything) yet **effective** (catching the most important regressions based on risk, impact, and traceability).